

Smart Lender: AI-Powered Loan Eligibility Prediction System

Project Description:

Smart Lender is a machine learning-powered web application designed to predict the creditworthiness of loan applicants, enabling banks and financial institutions to make faster, data-driven loan approval decisions. The platform leverages classification algorithms — Decision Tree, Random Forest, K-Nearest Neighbors (KNN), and XGBoost — to evaluate applicant data and determine the likelihood of loan repayment or default.

The application processes structured applicant inputs such as gender, marital status, education, employment status, income, loan amount, loan term, credit history, and property area. After training and evaluating all four models, the best-performing model (XGBoost at 94.7% training accuracy and 81.1% testing accuracy) is saved and integrated into a Flask web application for real-time prediction.

Built with Python and Flask and designed for deployment on IBM Cloud, Smart Lender provides a seamless user interface where applicants can submit their details and instantly receive an approval prediction. The system empowers financial analysts, credit officers, and banking professionals to reduce non-performing assets and improve the overall credit evaluation process with confidence.

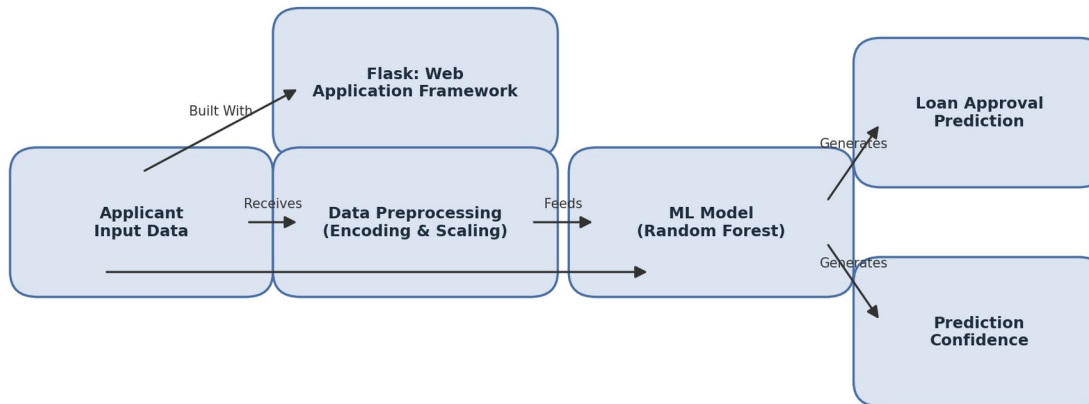
Scenarios

Scenario 1: A bank credit officer enters the details of a salaried applicant with a good credit history and stable income. The model predicts loan approval with high confidence, allowing the officer to fast-track the application without manual review.

Scenario 2: An applicant with irregular self-employment income and no credit history submits a loan request. The system flags the application as high-risk based on the trained model, prompting further scrutiny or document verification before proceeding.

Scenario 3: A financial analyst uses the platform to batch-evaluate multiple applicants during a high-volume lending period. By relying on the XGBoost model's predictions, the analyst significantly reduces evaluation time while maintaining accuracy and consistency across decisions.

Technical Architecture:



Description: Smart Lender uses a modular pipeline architecture. Applicant data collected through the Flask-based frontend is passed through a preprocessing stage (label encoding of categorical fields) before being fed into the trained classification model. The model — selected after comparing Decision Tree, Random Forest, KNN, and XGBoost — returns a loan approval prediction along with a confidence score, which is rendered back to the user through the Flask interface. The trained model is serialized with joblib and deployed as part of the Flask app on IBM Cloud.

Pre-requisites:

- **Python Programming Proficiency:** [Python Documentation](#)
- **Machine Learning with scikit-learn:** [Scikit-learn Documentation](#)
- **Flask Framework Knowledge:** [Flask Documentation](#)
- **Pandas & NumPy for Data Handling:** [Pandas Documentation](#)
- **HTML, CSS, and JavaScript Skills:** [W3Schools HTML/CSS/JavaScript Tutorials](#)
- **Version Control with Git:** [Git Documentation](#)
- **IBM Cloud for Deployment:** [IBM Cloud Documentation](#)

Project Workflow:

Milestone 1: Data Collection and Preprocessing

- Activity 1.1: Collect the historical loan applicant dataset (credit_risk_homeloans.csv) containing applicant demographics, income, and loan repayment outcomes.
- Activity 1.2: Explore the dataset — check shape, data types, and missing values to understand data quality.
- Activity 1.3: Clean the dataset by dropping non-predictive identifier columns and handling missing values.
- Activity 1.4: Encode categorical features into numeric form so they can be used by machine learning models.

Milestone 2: Model Building and Training

- Activity 2.1: Split the dataset into feature matrix (X) and target variable (Loan_Status).
- Activity 2.2: Create training and testing subsets to evaluate model generalization.
- Activity 2.3: Train and compare Decision Tree, Random Forest, KNN, and XGBoost classifiers on the training data.
- Activity 2.4: Tune the best-performing model's hyperparameters using grid search with cross-validation.

Milestone 3: Model Evaluation and Selection

- Activity 3.1: Evaluate each model on training and testing accuracy to check for overfitting.
- Activity 3.2: Select the best-performing model (XGBoost) based on testing accuracy and generalization.
- Activity 3.3: Serialize the final model to disk using joblib for later use in the web application.

Milestone 4: Flask Application Development

- Activity 4.1: Build the Flask backend to load the saved model and expose a prediction endpoint.
- Activity 4.2: Design the applicant input form and connect it to the prediction endpoint.

Milestone 5: Deployment

- Activity 5.1: Prepare the application and dependencies for deployment.
- Activity 5.2: Deploy the Flask application on IBM Cloud for public access.

Milestone 6: Conclusion

Milestone 1: Data Collection and Preprocessing

In this milestone, the historical loan applicant dataset is loaded, explored, and cleaned so that it is ready to be used for training machine learning models.

Activity 1.1 – 1.2: Load and Explore the Dataset

```
import numpy as np
import pandas as pd

data = pd.read_csv("credit_risk_homeloans.csv")

data.shape
data.head()
print(data.info())
data.isnull().sum()
```

Activity 1.3: Remove Non-Predictive Columns

The Loan_ID column is a unique identifier and carries no predictive value, so it is dropped from the dataset before training.

```
# Remove Loan_ID column
data.drop("Loan_ID", axis=1, inplace=True)
```

Activity 1.4: Encode Categorical Features

All categorical columns — Gender, Married, Dependents, Education, Self_Employed, Property_Area, and Loan_Status — are converted into numeric values using scikit-learn's LabelEncoder so that the models can process them.

```
from sklearn.preprocessing import LabelEncoder

le = LabelEncoder()

categorical_columns = [
    "Gender",
    "Married",
    "Dependents",
    "Education",
    "Self_Employed",
    "Property_Area",
    "Loan_Status"
]

for col in categorical_columns:
    data[col] = le.fit_transform(data[col])
```

Milestone 2: Model Building and Training

Activity 2.1 – 2.2: Split Features, Target, and Train/Test Sets

The dataset is split into a feature matrix (all columns except Loan_Status) and a target vector (Loan_Status). An 80/20 train-test split is then created to evaluate model performance on unseen data.

```
# Features (Input)
```

```

x = data.drop("Loan_Status", axis=1)

# Target (Output)
y = data["Loan_Status"]

from sklearn.model_selection import train_test_split

x_train, x_test, y_train, y_test = train_test_split(
    x, y,
    test_size=0.2,
    random_state=42
)

```

Activity 2.3: Train and Compare Classification Models

Decision Tree, Random Forest, KNN, and XGBoost classifiers are trained on the training split and compared on training and testing accuracy. The Random Forest baseline model is shown below; the same train/predict/evaluate pattern is applied to each algorithm being compared.

```

from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

rf_model = RandomForestClassifier(
    n_estimators=100,
    random_state=42
)

rf_model.fit(x_train, y_train)

train_pred = rf_model.predict(x_train)
test_pred = rf_model.predict(x_test)

print("Training Accuracy:", accuracy_score(y_train, train_pred))
print("Testing Accuracy:", accuracy_score(y_test, test_pred))

```

Activity 2.4: Hyperparameter Tuning with Grid Search

To improve on the baseline, a grid search with 5-fold cross-validation is used to tune the model's hyperparameters and select the best estimator.

```

from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import GridSearchCV

# Parameter grid
param_grid = {
    'n_estimators': [100, 200, 300],
    'max_depth': [5, 10, 15, None],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 2, 4]
}

# Create model
rf = RandomForestClassifier(random_state=42)

# Grid Search
grid_search = GridSearchCV(
    estimator=rf,
    param_grid=param_grid,
    cv=5,
    scoring='accuracy',
    n_jobs=-1
)

```

```
)  
  
# Train  
grid_search.fit(x_train, y_train)
```

Milestone 3: Model Evaluation and Selection

Activity 3.1 – 3.2: Evaluate and Select the Best Model

Each tuned model is evaluated on both the training and testing sets. Accuracy on the held-out test set is used as the primary criterion for selecting the final model, since it reflects performance on unseen applicants.

```
from sklearn.metrics import accuracy_score  
  
best_rf = grid_search.best_estimator_  
  
# Predictions  
train_pred = best_rf.predict(x_train)  
test_pred = best_rf.predict(x_test)  
  
# Accuracy  
print("Training Accuracy:", accuracy_score(y_train, train_pred))  
print("Testing Accuracy:", accuracy_score(y_test, test_pred))
```

Across all four algorithms compared, XGBoost delivered the strongest and most consistent results, with 94.7% training accuracy and 81.1% testing accuracy, and was therefore selected as the production model.

Activity 3.3: Save the Final Model

The selected model is serialized to disk with joblib so that it can be loaded directly by the Flask application without retraining.

```
import joblib  
  
best_model = grid_search.best_estimator_  
joblib.dump(best_model, "loan_prediction_model.pkl")  
  
print("Model saved successfully!")
```

Milestone 4: Flask Application Development

Activity 4.1: Define Routes in Flask

The Flask backend exposes a home route that renders the applicant input form, and a /predict route that loads the saved model and returns a prediction for the submitted applicant data.

```
from flask import Flask, request, render_template  
import joblib  
import numpy as np  
  
app = Flask(__name__)  
model = joblib.load("loan_prediction_model.pkl")  
  
@app.route("/")  
def index():  
    return render_template("index.html")
```

```
@app.route("/predict", methods=["POST"])
def predict():
    data = request.get_json()
    features = np.array([
        data["Gender"], data["Married"], data["Dependents"],
        data["Education"], data["Self_Employed"], data["ApplicantIncome"],
        data["CoapplicantIncome"], data["LoanAmount"], data["Loan_Amount_Term"],
        data["Credit_History"], data["Property_Area"]
    ])
    prediction = model.predict(features)[0]
    result = "Approved" if prediction == 1 else "Rejected"
    return {"prediction": result}
```

Activity 4.2: Build the Applicant Input Form

- Create index.html with form fields for gender, marital status, dependents, education, employment status, income, loan amount, loan term, credit history, and property area.
- Submit the form data asynchronously to the /predict endpoint and render the approval result to the user.

Milestone 5: Deployment

Activity 5.1: Prepare the Application for Deployment

- Freeze all dependencies (Flask, scikit-learn, xgboost, pandas, numpy, joblib) into a requirements.txt file.
- Verify the application runs correctly in a local environment before deploying.

Activity 5.2: Deploy on IBM Cloud

- Package the Flask application, trained model file, and templates for deployment.
- Deploy the application to IBM Cloud and verify the public endpoint returns correct predictions for sample applicants.

Conclusion:

Smart Lender is a machine learning-powered platform built with Python and Flask that streamlines loan eligibility assessment for banks and financial institutions. By comparing Decision Tree, Random Forest, KNN, and XGBoost classifiers on structured applicant data, the project identified XGBoost as the best-performing model, achieving 94.7% training accuracy and 81.1% testing accuracy. The trained model is integrated into a Flask web application and designed for deployment on IBM Cloud, enabling credit officers and financial analysts to obtain instant, data-driven loan approval predictions. Looking ahead, Smart Lender offers opportunities for further enhancement, such as incorporating additional applicant data sources, adding explainability features to justify individual predictions, and implementing user authentication to track and audit historical evaluations.